ObserveML — Security & Scalability Audit

🔐 SECURITY ISSUES

S1 · JWT Token Exposed in URL (High)

File: `useAlerts.js`

```
js

// ❌ Token in query string — logged by every proxy, browser history, server log
const ws = new WebSocket(`ws://localhost:8000/ws/alerts?token=${token}`)
```

WebSocket tokens in URLs are visible in:

- Server access logs
- Browser history
- Any reverse proxy (nginx, Cloudflare)
- CDN logs

Fix: Pass the token in the first WebSocket message after connection:

```
js

ws.onopen = () => {
  ws.send(JSON.stringify({ type: 'auth', token }))
}
// Backend: in websocket_alerts(), wait for first message, validate token, then proceed
```

S2 · SMTP Password Stored in Plaintext (High)

File: `users.py` (route) → `user_settings` table

The Settings page sends `smtpPassword` to `PUT /api/users/me/settings`, which stores it as a plain string in the `user_settings` table. Anyone with DB read access (DBA, backup restore, SQL injection) can see it.

Fix:

1. Encrypt it server-side before storage using Fernet (symmetric encryption) with a key derived from `SECRET_KEY`
2. Never return it in `GET /api/users/me/settings` — only return a flag `_smtpPasswordSaved: true`

The improved `Settings.jsx` already implements the frontend side (write-only password field). The backend needs:

```python
from cryptography.fernet import Fernet
# Encrypt before save, decrypt only when sending email, never return in GET
```

S3 · In-Memory Rate Limiter Leaks Memory (Medium)

File: `auth.py`

```python
_rate_limit_store: dict[str, list] = {}
```

Old entries are never evicted. After weeks of traffic, this dict grows unboundedly. Also, it resets on every restart — a deliberate restart floods login attempts.

Fix:

1. Use `slowapi` (already in `main.py`) consistently on `/api/auth/login` and `/api/auth/register` — it handles cleanup automatically.
2. If keeping the custom implementation, add a TTL eviction:

```python
# Prune entries older than RATE_LIMIT_WINDOW on every call
_rate_limit_store[key] = [t for t in hits if now - t < RATE_LIMIT_WINDOW]
```

S4 · Admin Role Check Is Fragile (Medium)

File: `metrics.py`

```python
```

```
def _is_admin(user: UserResponse) -> bool:
    return str(user.role) in ("admin", "Role.admin")  # ❌ "Role.admin" is an enum repr artifact
```

This exists because `user.role` is a `Role` enum, and `str(Role.admin)` → `"Role.admin"` in some Python versions. This is a latent bug.

Fix:

```
python

def _is_admin(user: UserResponse) -> bool:
    return user.role == Role.admin  # direct enum comparison — no stringly-typed logic
```

S5 · No HTTPS / Secure Cookie Enforcement (Medium)

File: `main.py`, `client.js`

All traffic is plain HTTP (`http://localhost:8000`). JWTs are stored in `localStorage` (XSS-accessible). For production:

Fixes:

1. Add `Strict-Transport-Security` header to the security middleware
2. Move tokens to `HttpOnly` cookies (requires backend `Set-Cookie` on login)
3. Add CSP header: `Content-Security-Policy: default-src 'self'`

S6 · SECRET_KEY Length Not Enforced (Low)

File: `config.py`

```
python

if not _secret_key:
    raise ValueError("SECRET_KEY not set")
# ❌ No minimum length check — "abc" passes
```

A weak secret key makes JWTs trivially forgeable.

Fix:

```
python
```

```
if not _secret_key or len(_secret_key) < 32:
    raise ValueError("SECRET_KEY must be at least 32 characters")
```

S7 · Audit Log Missing for API Key Usage (Low)

File: `metrics.py`

When an API key is used to ingest metrics, `key.last_used` is updated but no `AuditLog` entry is written. If a key is compromised, there's no record of what was ingested with it.

Fix: Add `log_action(db, key.owner_id, key.owner_email, "apikey.ingest", ...)` after a successful key auth in `_resolve_ingest_auth`.

📈 SCALABILITY ISSUES

SC1 · WebSocket Connections Are Single-Process (Critical)

File: `alert_service.py`

```
python

class ConnectionManager:
    def __init__(self):
        self.active: List[WebSocket] = []   # ❌ in-memory list
```

If you run 2+ Uvicorn workers (standard production config: `uvicorn app.main:app --workers 4`), each worker has its own `active` list. A message ingested on worker 1 only broadcasts to clients connected to worker 1.

The Redis Pub/Sub fallback exists but is only activated on first WebSocket connect — not guaranteed.

Fix: Make Redis the default, not the fallback:

```
python

# In connect(), always initialize Redis if REDIS_URL is set
# Or use a proper WS broadcast library like `broadcaster`
```

SC2 · In-Memory Alert Cooldown Doesn't Work Multi-Process (High)

File: `alert_service.py`

```python
_alerted: dict[tuple, datetime] = {}  # ❌ per-process — each worker fires its own alerts
```

With 4 workers, every threshold breach fires 4 duplicate alerts (one per worker), each bypassing the cooldown that only exists in that process's memory.

Fix: Store cooldown state in Redis:

```python
async def _can_alert_redis(key: tuple) -> bool:
    if not redis_client: return _can_alert(key, datetime.now(timezone.utc))
    lock_key = f"alert_cooldown:{key[0]}:{key[1]}"
    result = await redis_client.set(lock_key, "1", nx=True, ex=COOLDOWN_SECONDS)
    return result is not None
```

SC3 · N+1 Query on Every Metric Ingest (High)

File: `metrics.py`, `alert_service.py`

Every call to `POST /api/metrics/ingest` triggers `_check_thresholds()`, which:

1. Queries all active ML models
2. For each ML model: queries latest 3 readings
3. Queries all active LLM models
4. For each LLM model: queries latest 3 readings

With 20 models and 5 ingests/second from the simulator: **100+ queries/second** just for threshold checking, all doing the same full-table scans.

Fix:

1. Only check the specific model being ingested — not all models:

```python
# In ingest_metric(), pass model_id + model_type to _check_thresholds
asyncio.ensure_future(_check_thresholds(model_id=body.model_id, model_type=body.model_type))
```

2. Keep the 30-second background sweep for models without active ingestion.

SC4 · Chart Bucketing Done in Python, Not SQL (Medium)

Files: `metrics.py` (routes), `metrics_services.py`

```python
rows = db.query(MetricReading).filter(...).all()  # ❌ load ALL rows into Python
# Then loop and bucket them in Python
for r in rows:
    bucket_min = (ts.minute // 30) * 30
    ...
```

Loading 1000+ readings into Python memory to average them is wasteful. This should be done in SQL:

```sql
SELECT
  DATE_TRUNC('hour', recorded_at) +
    INTERVAL '30 min' * FLOOR(EXTRACT(minute FROM recorded_at) / 30) AS bucket,
  metric_name,
  AVG(metric_value) AS avg_value
FROM metric_readings
WHERE model_id = :id AND recorded_at >= :since
GROUP BY bucket, metric_name
ORDER BY bucket;
```

SC5 · No Index on `metric_readings.model_type` Alone (Medium)

File: `metric_reading.py`

```python
__table_args__ = (
    Index("ix_metric_model_time", "model_id", "model_type", "recorded_at"),
)
```

The composite index `(model_id, model_type, recorded_at)` is good for per-model queries. But `_ml_chart_from_db` and `_llm_chart_from_db` filter by `model_type` + `model_ids IN (...)` — the IN clause on `model_id` with many IDs may not use the index effectively.

Fix: Add a partial index for large deployments:

```python
Index("ix_metric_type_time", "model_type", "recorded_at"),
```

SC6 · Rate Limiter Not Distributed (Medium)

File: `auth.py`

Already noted in Security (S3), but also a scalability issue: with multiple workers, each has its own `_rate_limit_store`, so an attacker gets `MAX_FAILED_ATTEMPTS × num_workers` attempts before being blocked.

Fix: Use `slowapi` with a Redis backend, or use the DB-level `failed_attempts` field (already exists on the `User` model) — which is process-safe since it's in the DB.

SC7 · `data_retention_job` Runs Full Table Delete (Low)

File: `alert_service.py`

```python
deleted = db.query(MetricReading).filter(MetricReading.recorded_at < cutoff).delete()
```

A bulk DELETE on a large `metric_readings` table with millions of rows will lock the table, spike I/O, and potentially cause write timeouts for ingestion during the purge window.

Fix: Batch-delete in chunks:

```python
while True:
    ids = db.query(MetricReading.id).filter(
        MetricReading.recorded_at < cutoff
    ).limit(1000).all()
    if not ids: break
    db.query(MetricReading).filter(
        MetricReading.id.in_([r.id for r in ids])
    ).delete(synchronize_session=False)
    db.commit()
    await asyncio.sleep(0.1)  # yield between batches
```

Summary

Security

#	Issue	Severity	File
S1	JWT in WebSocket URL	High	useAlerts.js
S2	SMTP password stored plaintext	High	users.py / DB
S3	Rate limiter memory leak + bypass on restart	Medium	auth.py
S4	Fragile string role check	Medium	metrics.py
S5	No HTTPS / HttpOnly cookies	Medium	main.py / client.js
S6	SECRET_KEY length not enforced	Low	config.py
S7	API key usage not in audit log	Low	metrics.py

Scalability

#	Issue	Severity	File
SC1	WebSocket broadcast not multi-process	Critical	alert_service.py
SC2	Alert cooldown not multi-process	High	alert_service.py
SC3	N+1 queries on every ingest	High	metrics.py / alert_service.py
SC4	Chart bucketing in Python not SQL	Medium	metrics_services.py
SC5	Missing index for cross-model chart queries	Medium	metric_reading.py
SC6	Rate limiter not distributed	Medium	auth.py
SC7	Bulk table delete during retention job	Low	alert_service.py